

Package ‘tpridge’

Reference Manual

Version 0.2.0

Type	Package
Title	Two-Parameter Ridge Estimation of HAR Realized-Volatility Models
Version	0.2.0
Depends	R ($\geq$ 4.1.0)
Imports	data.table ($\geq$ 1.14.0), methods, stats, utils, graphics, grDevices
Suggests	ggplot2, sandwich, lmtest, MCS, rugarch, future, future.apply, testthat, knitr, rmarkdown
License	GPL-3
Encoding	UTF-8
URL	https://github.com/talhaomer472/tpridge
Description	Estimation, forecasting and evaluation for heterogeneous autoregressive (HAR) models of realized volatility using the two-parameter ridge estimator of Lipovetsky and Conklin (2005). The package covers the full pipeline: downloading five-minute klines directly from Binance, constructing realized measures, estimating ordinary least squares and the one- and two-parameter ridge estimators on rolling windows, evaluating forecasts with Diebold-Mariano tests, the model confidence set and Value-at-Risk backtests, and reproducing the Monte Carlo design of the accompanying paper. The estimator is not specific to volatility models: a formula interface, <code>tpr_lm()</code> , applies it to any regression, and the penalty and the two-parameter scalar are exported as standalone functions so they can be used with any design matrix.

R topics documented:

<code>binance_klines</code>	3
<code>binance_symbols</code>	4
<code>coef.tpr_fit</code>	5
<code>fitted.tpr_fit</code>	6
<code>har_design</code>	7
<code>har_features</code>	8
<code>har_simulate</code>	9
<code>plot.tpr_fit</code>	10
<code>plot.tpr_mc</code>	11
<code>plot.tpr_rolling</code>	12
<code>predict.tpr_fit</code>	13
<code>predict.tpr_lm</code>	14
<code>print.summary.tpr_lm</code>	15
<code>print.tpr_fit</code>	16
<code>print.tpr_lm</code>	17
<code>print.tpr_mc</code>	18
<code>print.tpr_rolling</code>	19
<code>realized_measures</code>	20
<code>residuals.tpr_lm</code>	21
<code>summary.tpr_lm</code>	22
<code>summary.tpr_rolling</code>	23
<code>tpr_dm</code>	24

tpr_eff_df	25
tpr_fit	26
tpr_lambda	27
tpr_lm	28
tpr_mcs	29
tpr_metrics	30
tpr_montecarlo	31
tpr_q	33
tpr_rolling	34
tpr_var_backtest	35
tpr_window	36

binance_klines

Download five-minute klines from Binance

Description

Downloads the monthly kline archives published at `data.binance.vision` and returns them as a single table. Nothing is required beyond an internet connection: the files are public and no API key is needed.

Two details matter for replication and are handled here. Binance switched the `open_time` field from milliseconds to microseconds partway through the history, which is detected and normalised. And some archives carry a header row while others do not, which is also detected.

Usage

```
binance_klines(symbol,  
  from,  
  to,  
  interval = "5m",  
  dir = file.path(tempdir(),  
    "binance"),  
  quiet = FALSE)
```

Arguments

<code>symbol</code>	Trading pair, for example "BTCUSDT".
<code>from</code>	First and last month, as "YYYY-MM".
<code>to</code>	First and last month, as "YYYY-MM".
<code>interval</code>	Kline interval. Default "5m".
<code>dir</code>	Directory in which to cache the downloaded archives. A month already present is not downloaded again. Default is a session temporary directory.
<code>quiet</code>	Suppress progress messages.

Value

A `data.table` with columns `open_time` (POSIXct, UTC), `open`, `high`, `low`, `close`, `volume`, `quote_volume`, `num_trades`, `taker_buy_base`, `taker_buy_quote`.

Examples

```
# one month of Bitcoin data  
k <- binance_klines("BTCUSDT", from = "2024-01", to = "2024-01")  
head(k)  
}
```

binance_symbols

Symbols used in the accompanying paper

Description

Usage

```
| binance_symbols()
```

Value

A character vector of Binance trading pairs.

Examples

```
| binance_symbols()
```

coef.tpr_fit

Coefficients of a fitted model

Description

Usage

```
| coef.tpr_fit(object, estimator = "Ridge-II", ...)
```

Arguments

<code>object</code>	An object of class <code>tpr_fit</code> .
<code>estimator</code>	One of "OLS", "Ridge-I" or "Ridge-II".
<code>...</code>	Ignored.

Value

A named numeric vector on the original scale, including the intercept.

Examples

```
| set.seed(1)
| f <- tpr_fit(matrix(rnorm(300), ncol = 3), rnorm(100))
| coef(f, "Ridge-I")
```

fitted.tpr_fit

Fitted values of a fitted model

Description

Usage

```
| fitted.tpr_fit(object, estimator = "Ridge-II", ...)
```

Arguments

<code>object</code>	An object of class <code>tpr_fit</code> .
<code>estimator</code>	One of "OLS", "Ridge-I" or "Ridge-II".
<code>...</code>	Ignored.

Value

A numeric vector of in-sample fitted values.

Examples

```
| set.seed(1)
| f <- tpr_fit(matrix(rnorm(300), ncol = 3), rnorm(100))
| head(fitted(f))
```

har_design

Design matrix and response from a feature table

Description

Usage

```
| har_design(features, predictors = NULL)
```

Arguments

<code>features</code>	Output of <code>[har_features()]</code> .
<code>predictors</code>	Character vector of column names to use. Default is every column other than <code>y</code> and <code>date</code> .

Value

A list with `X`, `y` and `date`, complete cases only.

Examples

```
| set.seed(1)
| d <- har_features(har_simulate(500, sigma2 = 0.5))
| des <- har_design(d)
| dim(des$X)
```

har_features

HAR features from a daily realized measure

Description

Builds the daily, weekly and monthly components of the HAR model as rolling averages ending at $t - 1$, together with the lead of the target, so that each row contains a forecast origin and the value it predicts. The logarithmic transformation is the default because a squared error criterion applied to realized variance in levels is dominated by a handful of days.

Usage

```
har_features(rv,  
  date = NULL,  
  lags = c(1,  
    5,  
    22),  
  log = TRUE,  
  extra = NULL)
```

Arguments

rv	Numeric vector of daily realized variance, in time order.
date	Optional vector of dates, carried through to the output.
lags	Averaging windows. Default <code>c(1, 5, 22)</code> , the daily, weekly and monthly components of Corsi (2009).
log	Return the logarithm of each component. Default <code>TRUE</code> .
extra	Optional <code>data.frame</code> of further predictors, aligned with <code>rv</code> , appended unchanged.

Value

A `data.table` whose first column `y` is the target for the next day, followed by the HAR components and any extras.

Examples

```
set.seed(1)  
rv <- har_simulate(500, sigma2 = 0.5)  
d <- har_features(rv)  
head(d)
```

har_simulate

Simulate a HAR process

Description

Generates $RV_t = \mu_t \eta_t$ with $\mu_t = \alpha + \beta_d RV_{t-1} + \beta_w \overline{RV}_{t-1}^{(w)} + \beta_m \overline{RV}_{t-1}^{(m)}$ and $\eta_t \sim \text{Gamma}(k, 1/k)$ with unit mean. Positivity is structural: no truncation or transformation is applied, and the conditional mean is exactly the HAR conditional mean.

Usage

```
har_simulate(n,  
             sigma2,  
             alpha = 0.1,  
             betas = c(0.3,  
                       0.3,  
                       0.3),  
             burn = 300L)
```

Arguments

n	Number of usable observations returned.
sigma2	Variance of the multiplicative shock, equal to $1/k$.
alpha	Intercept, strictly positive.
betas	The three persistence parameters, non-negative and summing to less than one.
burn	Burn-in discarded before the returned sample.

Value

A numeric vector of length `n + 22`, so that `[har_features()]` yields exactly `n` usable rows.

Examples

```
set.seed(1)  
rv <- har_simulate(1000, sigma2 = 0.5)  
c(mean = mean(rv), min = min(rv))
```

plot.tpr_fit

Plot a fitted model

Description

Usage

```
| plot.tpr_fit(x, which = c("path", "coef", "gap"), n_grid = 40L, ...)
```

Arguments

x	An object of class <code>tpr_fit</code> .
which	One of "path", the coefficient of determination of all three estimators against the effective degrees of freedom; "coef", the standardized coefficients; or "gap", the fit gain of Proposition 2 against the penalty.
n_grid	Number of penalty values on the path.
...	Passed to the underlying plotting calls.

Value

Invisibly, the data behind the plot.

Examples

```
| set.seed(1)
| X <- matrix(rnorm(300), ncol = 3)
| y <- as.numeric(X %*% c(1, 0.5, 0.2)) + rnorm(100)
| plot(tpr_fit(X, y), which = "path")
```

plot.tpr_mc

Plot a Monte Carlo study

Description

Usage

```
| plot.tpr_mc(x, which = c("r2", "recovery", "identity"), ...)
```

Arguments

<code>x</code>	An object of class <code>tpr_mc</code> .
<code>which</code>	One of "r2", the out-of-sample fit by sample size; "recovery", the share of the sacrificed fit that the second parameter restores; or "identity", the empirical fit gain against its theoretical value.
<code>...</code>	Passed to the underlying plotting calls.

Value

Invisibly, the data behind the plot.

Examples

```
| mc <- tpr_montecarlo(n = c(100, 200), sigma2 = 0.5, reps = 20, seed = 1)
| plot(mc, which = "identity")
| }
```

plot.tpr_rolling

Plot rolling forecasts

Description

Usage

```
plot.tpr_rolling(x,
  which = c("forecast",
    "q",
    "lambda",
    "path"),
  estimator = "Ridge-II",
  log = TRUE,
  ...)
```

Arguments

x	An object of class <code>tpr_rolling</code> .
which	One of "forecast", the realization against the forecast; "q", the two-parameter scalar over time; "lambda", the penalty and the effective degrees of freedom; or "path", the out-of-sample fit against the effective degrees of freedom, which requires the object to have been produced with <code>rule = "df"</code> .
estimator	Estimator to display where only one is shown.
log	Plot on a logarithmic vertical scale.
...	Passed to the underlying plotting calls.

Value

Invisibly, the data behind the plot.

Examples

```
set.seed(1)
des <- har_design(har_features(har_simulate(600, sigma2 = 0.5)))
r <- tpr_rolling(des$X, des$y, window = 250)
plot(r, which = "q")
```

predict.tpr_fit

Forecast from a fitted model

Description

Usage

```
predict.tpr_fit(object,  
  newdata,  
  estimator = "Ridge-II",  
  level = FALSE,  
  ...)
```

Arguments

<code>object</code>	An object of class <code>tpr_fit</code> .
<code>estimator</code>	One of "OLS", "Ridge-I" or "Ridge-II".
<code>...</code>	Ignored.
<code>newdata</code>	Matrix of new regressor values, with the same columns as the design used to fit.
<code>level</code>	Return the forecast on the level scale, applying $\exp(\hat{f} + \hat{\sigma}^2/2)$. Appropriate only when the model was fitted to a logarithm.

Value

A numeric vector of forecasts.

Examples

```
set.seed(1)  
X <- matrix(rnorm(300), ncol = 3); y <- rnorm(100)  
f <- tpr_fit(X[1:90, ], y[1:90])  
predict(f, X[91:100, ])
```

predict.tpr_lm

Description

Usage

```
| predict.tpr_lm(object, newdata, estimator = "Ridge-II", ...)
```

print.summary.tpr_lm

Description

Usage

```
| print.summary.tpr_lm(x, ...)
```

print.tpr_fit

Print a fitted model

Description

Usage

```
| print.tpr_fit(x, ...)
```

Arguments

x	An object of class <code>tpr_fit</code> .
...	Ignored.

Value

x, invisibly.

Examples

```
| set.seed(1)  
| print(tpr_fit(matrix(rnorm(300), ncol = 3), rnorm(100)))
```

print.tpr_lm

Description

Usage

```
| print.tpr_lm(x, ...)
```

print.tpr_mc

Print a Monte Carlo study

Description

Usage

```
| print.tpr_mc(x, ...)
```

Arguments

x	An object of class <code>tpr_mc</code> .
...	Ignored.

Value

x, invisibly.

Examples

```
| print(tpr_montecarlo(n = 150, sigma2 = 0.5, reps = 5, progress = FALSE))  
| }
```

print.tpr_rolling

Print rolling forecasts

Description

Usage

```
| print.tpr_rolling(x, ...)
```

Arguments

x	An object of class <code>tpr_rolling</code> .
...	Ignored.

Value

x, invisibly.

Examples

```
| set.seed(1)
| des <- har_design(har_features(har_simulate(400, 0.5)))
| print(tpr_rolling(des$X, des$y, window = 150))
```

realized_ measures

Daily realized measures from intraday prices

Description

Aggregates intraday closes into the daily measures used by the HAR literature. Returns are differenced over the full series rather than within each day, so a complete day contributes one return per interval including the one spanning midnight. This is the right convention for a market that trades continuously; set `within_day = TRUE` to exclude it, at the cost of one return per day.

Usage

```
realized_measures(x,
  bars_per_day = 288L,
  min_share = 0.9,
  within_day = FALSE)
```

Arguments

<code>x</code>	A table with an <code>open_time</code> column of class <code>POSIXct</code> and a <code>close</code> column, as returned by <code>[binance_klines()]</code> . Volume columns are used when present.
<code>bars_per_day</code>	Expected number of intervals in a full day. For five-minute data this is 288.
<code>min_share</code>	Days with fewer than this share of the expected intervals have their measures set to NA. Default 0.9.
<code>within_day</code>	Compute returns within each calendar day only. Default <code>FALSE</code> .

Value

A `data.table` with one row per day: `date`, `n_5m`, `complete_day`, `RV`, `BV`, `Jump`, `RSV_pos`, `RSV_neg`, `RQ`, `RSkew`, `RKurt`, `Parkinson`, `ret_d`, `abs_ret_d`, `close`, and, when the inputs are present, `log_volume`, `num_trades`, `taker_buy_share`, `amihud_proxy`.

Examples

```
k <- binance_klines("BTCUSDT", "2024-01", "2024-02")
rm_ <- realized_measures(k)
head(rm[, .(date, RV, BV, Jump)])
}
```

residuals.tpr_lm

Description

Usage

```
| residuals.tpr_lm(object, estimator = "Ridge-II", ...)
```

summary.tpr_lm

Summarise a two-parameter ridge regression

Description

Reports the coefficients of all three estimators side by side, together with the diagnostics that bear on whether shrinkage is called for: the variance inflation factors, the condition number of the design, the penalty, the scalar q and the effective degrees of freedom.

No standard errors are given. The sampling distribution of a shrinkage estimator is not that of least squares, so the usual standard errors and the tests built on them do not carry over. Inference for the ridge estimators would require a bootstrap.

Usage

```
| summary.tpr_lm(object, ...)
```

Arguments

<code>object</code>	An object of class <code>tpr_lm</code> .
<code>...</code>	Ignored.

Value

An object of class `summary.tpr_lm`, invisibly printed.

Examples

```
| data(longley)  
| summary(tpr_lm(Employed ~ GNP + Unemployed + Year, data = longley))
```

summary.tpr_rolling

Summarise rolling forecasts

Description

Usage

```
| summary.tpr_rolling(object, ...)
```

Arguments

<code>object</code>	An object of class <code>tpr_rolling</code> .
<code>...</code>	Ignored.

Value

A `data.table` with one row per penalty rule and estimator, giving the mean squared error, the out-of-sample coefficient of determination against the prevailing mean, the mean in-sample fit, the median penalty, and the mean values of `q` and the effective degrees of freedom.

Examples

```
| set.seed(1)
| des <- har_design(har_features(har_simulate(400, 0.5)))
| summary(tpr_rolling(des$X, des$y, window = 150))
```

tpr_dm

Diebold and Mariano test with a HAC long-run variance

Description

A positive statistic means the first loss series is the larger, that is, the second model forecasts better.

Usage

```
| tpr_dm(loss_a, loss_b, lag = NULL)
```

Arguments

loss_a	Loss series of equal length.
loss_b	Loss series of equal length.
lag	Truncation lag. NULL uses $\lfloor 4(n/100)^{2/9} \rfloor$.

Value

A named vector with the statistic, its p value and the lag.

Examples

```
| set.seed(1); la <- rchisq(300, 2); lb <- rchisq(300, 2)  
| tpr_dm(la, lb)
```

tpr_eff_df

Effective degrees of freedom of the ridge operator

Description

The quantity $\text{df}(\lambda) = \sum_j e_j / (e_j + \lambda)$, where e_j are the eigenvalues of $X'X$. It falls from p at $\lambda = 0$ towards zero as the penalty grows, and is comparable across sample sizes and designs in a way that λ itself is not.

Usage

```
| tpr_eff_df(x, lambda, standardize = TRUE)
```

Arguments

x	Either a numeric matrix of regressors, or a vector of eigenvalues of the cross-product matrix.
lambda	Non-negative penalty.
standardize	Logical. If x is a matrix, standardize its columns before forming the cross-product. Default TRUE .

Value

A single number, the effective degrees of freedom.

Examples

```
| set.seed(1)
| X <- matrix(rnorm(300), ncol = 3)
| tpr_eff_df(X, lambda = 0)
| tpr_eff_df(X, lambda = 1e6)
```

tpr_fit

Fit OLS, one-parameter ridge and two-parameter ridge

Description

All three estimators come from a single decomposition of the design, so the comparison between them is exact. The same λ enters the one-parameter estimator and the computation of q , which is required for q to maximise the fit along the ray.

Usage

```
tpr_fit(X,  
  y,  
  lambda = NULL,  
  rule = c("hkb",  
    "df"),  
  target_df = NULL,  
  standardize = TRUE)
```

Arguments

lambda Penalty. If NULL it is selected by **rule**.

Value

An object of class `tpr_fit`.

Examples

```
set.seed(1)  
X <- matrix(rnorm(300), ncol = 3)  
y <- as.numeric(X %*% c(1, 0.5, 0.2)) + rnorm(100)  
f <- tpr_fit(X, y)  
f  
coef(f, "Ridge-II")  
abs(f$gap_empirical - f$gap_theoretical) < 1e-12
```

tpr_lambda

Ridge penalty

Description

Two selection rules. "hkb" is the data-driven rule of Hoerl, Kennard and Baldwin (1975), $\hat{\lambda} = p\hat{\sigma}^2/\hat{\beta}'_{OLS}\hat{\beta}_{OLS}$. "df" solves $\text{df}(\lambda) = \text{target_df}$ by one-dimensional root finding, which lets the user state the amount of shrinkage in interpretable units.

Usage

```
tpr_lambda(X,
  y,
  rule = c("hkb",
    "df"),
  target_df = NULL,
  standardize = TRUE)
```

Arguments

X	Numeric matrix of regressors, without an intercept column.
y	Numeric response.
rule	Either "hkb" or "df".
target_df	Target effective degrees of freedom, required when rule = "df". Must lie in $(0, \text{ncol}(X)]$.
standardize	Logical. Standardize the columns of X and centre y first. Default TRUE, and recommended: the penalty is not scale invariant.

Value

A single non-negative number.

Examples

```
set.seed(1)
X <- matrix(rnorm(300), ncol = 3)
y <- as.numeric(X %*% c(1, 0.5, 0.2)) + rnorm(100)
tpr_lambda(X, y)
tpr_lambda(X, y, rule = "df", target_df = 1.5)
```

tpr_lm

Two-parameter ridge with a formula

Description

A formula interface to `[tpr_fit()]`, so the estimator can be used on any regression in the way `lm()` is used. Nothing about the two-parameter ridge is specific to volatility models: it is a general remedy for an ill-conditioned design.

Usage

```
tpr_lm(formula,
       data,
       lambda = NULL,
       rule = c("hkb",
               "df"),
       target_df = NULL,
       subset = NULL,
       na.action = stats::na.omit,
       standardize = TRUE)
```

Arguments

<code>formula</code>	A model formula, as for <code>[stats::lm()]</code> .
<code>data</code>	A data frame, list or environment holding the variables.
<code>lambda</code>	Penalty. If <code>NULL</code> it is chosen by <code>rule</code> .
<code>rule</code>	Either <code>"hkb"</code> , the data-driven rule of Hoerl, Kennard and Baldwin, or <code>"df"</code> , which solves for the penalty delivering <code>target_df</code> effective degrees of freedom.
<code>target_df</code>	Target effective degrees of freedom, used when <code>rule = "df"</code> .
<code>subset</code>	Optional vector of rows to use.
<code>na.action</code>	How to treat missing values. Defaults to <code>[stats::na.omit()]</code> .
<code>standardize</code>	Standardize the regressors before estimation. Default <code>TRUE</code> , and recommended: the penalty is not scale invariant.

Value

An object of class `tpr_lm`, inheriting from `tpr_fit`, with the call, terms and model frame attached so that `predict()` and `summary()` behave as expected.

See Also

`[tpr_fit()]` for the matrix interface, `[summary.tpr_lm()]`

Examples

```
## The classic ill-conditioned design of Longley (1967)
data(longley)
m <- tpr_lm(Employed ~ GNP + Unemployed + Armed.Forces +
           Population + Year, data = longley)

m
summary(m)
coef(m, "Ridge-II")
```

tpr_mcs

Model confidence set

Description

A thin wrapper around `MCS::MCSprocedure` that accepts a matrix of losses and returns a tidy table, extracting the result slot across the versions of that package.

Usage

```
| tpr_mcs(loss, alpha = 0.10, B = 5000L, statistic = "Tmax")
```

Arguments

<code>loss</code>	Matrix of losses, one column per model.
<code>alpha</code>	Size of the test. Default 0.10.
<code>B</code>	Bootstrap replications. Default 5000.
<code>statistic</code>	Either "Tmax" or "TR".

Value

A `data.table`, or `NULL` if the procedure fails.

Examples

```
| set.seed(1)
| L <- cbind(a = rchisq(300, 2), b = rchisq(300, 2.2), c = rchisq(300, 3))
| tpr_mcs(L)
| }
```

tpr_metrics

Forecast loss measures

Description

Usage

```
| tpr_metrics(actual, pred, benchmark = NULL, level = FALSE)
```

Arguments

actual	Numeric vector of realizations.
pred	Numeric vector of forecasts.
benchmark	Benchmark forecast used for the coefficient of determination. Default is the prevailing mean of actual .
level	Compute the quasi-likelihood loss, which requires both arguments to be strictly positive. Default FALSE .

Value

A one-row `data.table`.

Examples

```
| set.seed(1); a <- rnorm(100); f <- a + rnorm(100, sd = 0.5)  
| tpr_metrics(a, f)
```

tpr__montecarlo

Monte Carlo study of the three estimators

Description

Reproduces the design of the accompanying paper. For each cell of the grid the process of `[har_simulate()]` is drawn afresh, split into an estimation and an evaluation part, and all three estimators are rolled forward. When several penalty targets are supplied they are applied to the same simulated path within a replication, so the comparison across penalties is paired.

Usage

```
tpr_montecarlo(n = c(100,
  200,
  400,
  500),
  sigma2 = c(0.1,
  0.4,
  0.5,
  0.6,
  0.7,
  0.8,
  1.0,
  1.1,
  1.2),
  reps = 1000L,
  train_frac = 0.4,
  rule = c("hkb",
  "df"),
  target_df = c(3,
  2.5,
  2,
  1.5,
  1,
  0.5),
  alpha = 0.1,
  betas = c(0.3,
  0.3,
  0.3),
  seed = NULL,
  progress = TRUE)
```

Arguments

<code>n</code>	Sample sizes to consider.
<code>sigma2</code>	Variances of the multiplicative shock.
<code>reps</code>	Replications per cell.
<code>train_frac</code>	Share of each sample used for estimation.
<code>rule</code>	Penalty rule, "hkb" or "df".
<code>target_df</code>	Targets used when <code>rule = "df"</code> .
<code>alpha</code>	Parameters of the data-generating process.
<code>betas</code>	Parameters of the data-generating process.
<code>seed</code>	Seed for reproducibility.
<code>progress</code>	Report each cell as it completes.

Value

An object of class `tpr_mc`, a `data.table` with one row per replication, cell and estimator.

Examples

```
mc <- tpr_montecarlo(n = 200, sigma2 = 0.5, reps = 20, seed = 1)
print(mc)
}
```

tpr_q

The two-parameter scalar q

Description

The scalar that maximises the coefficient of determination along the ray $\{\hat{q}\hat{\beta}_I\}$, namely $q = Q_1/Q_2$ with $Q_1 = \hat{\beta}_I' r$ and $Q_2 = \hat{\beta}_I' G \hat{\beta}_I$. It exceeds one for every $\lambda > 0$, since $q = 1 + \lambda \|\hat{\beta}_I\|^2 / \|X \hat{\beta}_I\|^2$.

Usage

```
tpr_q(X,
      y,
      lambda = NULL,
      rule = c("hkb",
               "df"),
      target_df = NULL,
      standardize = TRUE)
```

Arguments

X	Numeric matrix of regressors, without an intercept column.
y	Numeric response.
rule	Either "hkb" or "df".
target_df	Target effective degrees of freedom, required when rule = "df". Must lie in $(0, \text{ncol}(X)]$.
standardize	Logical. Standardize the columns of X and centre y first. Default TRUE , and recommended: the penalty is not scale invariant.
lambda	Penalty. If NULL it is selected by rule .

Value

A single number greater than one, carrying attributes **lambda**, **rho** and **eff_df**.

Examples

```
set.seed(1)
X <- matrix(rnorm(300), ncol = 3)
y <- as.numeric(X %*% c(1, 0.5, 0.2)) + rnorm(100)
q <- tpr_q(X, y, lambda = 10)
q > 1
attr(q, "eff_df")
```

tpr_rolling

Rolling one-step-ahead forecasts

Description

Refits all three estimators in every window and returns one row per forecast origin per estimator. The window is decomposed once and every penalty rule requested is evaluated on that decomposition, which is what makes a path over several penalties inexpensive.

Usage

```
tpr_rolling(X,
  y,
  window,
  date = NULL,
  rule = c("hkb",
    "df"),
  target_df = NULL,
  log_target = TRUE,
  progress = FALSE)
```

Arguments

X	Numeric matrix of regressors.
y	Numeric response, aligned with the rows of X .
window	Width of the rolling estimation window.
date	Optional vector of dates for the rows of X .
rule	Penalty rule, "hkb" or "df".
target_df	Either a single target or a vector of targets, used when rule = "df". When several are supplied, all are applied to the same window, so the comparison across penalties is paired.
log_target	The response is on the logarithmic scale, so forecasts are also returned on the level scale with the correction $\exp(\hat{f} + \hat{\sigma}^2/2)$. Default TRUE .
progress	Print progress every 500 windows.

Value

An object of class **tpr_rolling**, a **data.table** with the forecast, the realization, the penalty, **q**, the effective degrees of freedom, the condition number, the in-sample fit and the two fit-gain quantities of Proposition 2.

Examples

```
set.seed(1)
d <- har_features(har_simulate(600, sigma2 = 0.5))
des <- har_design(d)
r <- tpr_rolling(des$X, des$y, window = 250)
summary(r)
```

tpr_var_backtest

Value-at-Risk backtests

Description

Builds $\widehat{\text{VaR}}_{t+1} = z_{\alpha} \sqrt{\hat{f}_{t+1}}$ from a variance forecast and tests the resulting violation sequence with the unconditional coverage test of Kupiec (1995) and the independence test of Christoffersen (1998).

Usage

```
tpr_var_backtest(variance_forecast,
  returns,
  alpha = 0.01,
  dist = c("normal",
    "student"),
  df = 5)
```

Arguments

<code>variance_forecast</code>	Numeric vector of one-step variance forecasts, strictly positive.
<code>returns</code>	Realized returns for the same days.
<code>alpha</code>	Tail probability, for example 0.01.
<code>dist</code>	Either "normal" or "student".
<code>df</code>	Degrees of freedom when <code>dist = "student"</code> . Default 5.

Value

A one-row `data.table` with the violation rate and both tests.

Examples

```
set.seed(1)
v <- rep(4e-4, 1000); r <- rnorm(1000, sd = sqrt(v))
tpr_var_backtest(v, r, alpha = 0.05)
```

tpr_window

Decompose a design once

Description

Standardizes the regressors, centres the response, and returns the sufficient statistics together with the eigenvalues, so that every penalty rule can be evaluated without repeating the expensive work.

Usage

```
| tpr_window(X, y, standardize = TRUE)
```

Arguments

X	Numeric matrix of regressors, without an intercept column.
y	Numeric response.
rule	Either "hkb" or "df".
target_df	Target effective degrees of freedom, required when rule = "df". Must lie in $(0, \text{ncol}(X)]$.
standardize	Logical. Standardize the columns of X and centre y first. Default TRUE , and recommended: the penalty is not scale invariant.

Value

A list of sufficient statistics.

Examples

```
| set.seed(1)
| w <- tpr_window(matrix(rnorm(300), ncol = 3), rnorm(100))
| round(w$eigenvalues, 2)
```